



CSE422 Lab Project Report

Project Title: Flat Price Classification

Section - 17

Group -L

Name	ID
Samura Huda	22201205
Tahsin Raihan Robbani	22201344

Table of Contents

1. Introduction	3
2. Dataset Description	3
○ Dataset Overview	
○ Correlation Analysis	
○ Imbalanced Dataset	
○ Exploratory Data Analysis	
3. Dataset Pre-processing	9
4. Dataset Splitting	11
5. Model Training & Testing	11
○ K-Nearest Neighbors (KNN)	
○ Logistic Regression	
○ Neural Network	
○ Decision Tree	
○ Naive Bayes	
○ K-Means Clustering (Unsupervised)	
6. Model Selection / Comparison Analysis	13
7. Conclusion	19

1. Introduction

The goal of this project is to build a system that can classify flats into Low, Medium, or High price using basic property information (size, rooms, floor, building age, distance to city center, and simple amenities). This is useful because quick price hints help students and families compare options and plan budgets.

We apply supervised learning (K-Nearest Neighbors, Logistic Regression, Neural Network, Decision Tree, Naive Bayes) and unsupervised learning (K-Means clustering) to this classification problem. By training on past data, the models can learn patterns that are not obvious at first glance.

Our motivation comes from real life: many flats look similar on paper, and price levels can be hard to judge. With machine learning, we can use the data to make fast, consistent predictions. Through this project, we have done preprocessing, feature encoding, scaling, model training, evaluation, and visualization. This helps us see how data preparation affects performance and how different models behave.

2. Dataset Description

Dataset Link: [flat_price_dataset.csv](#)

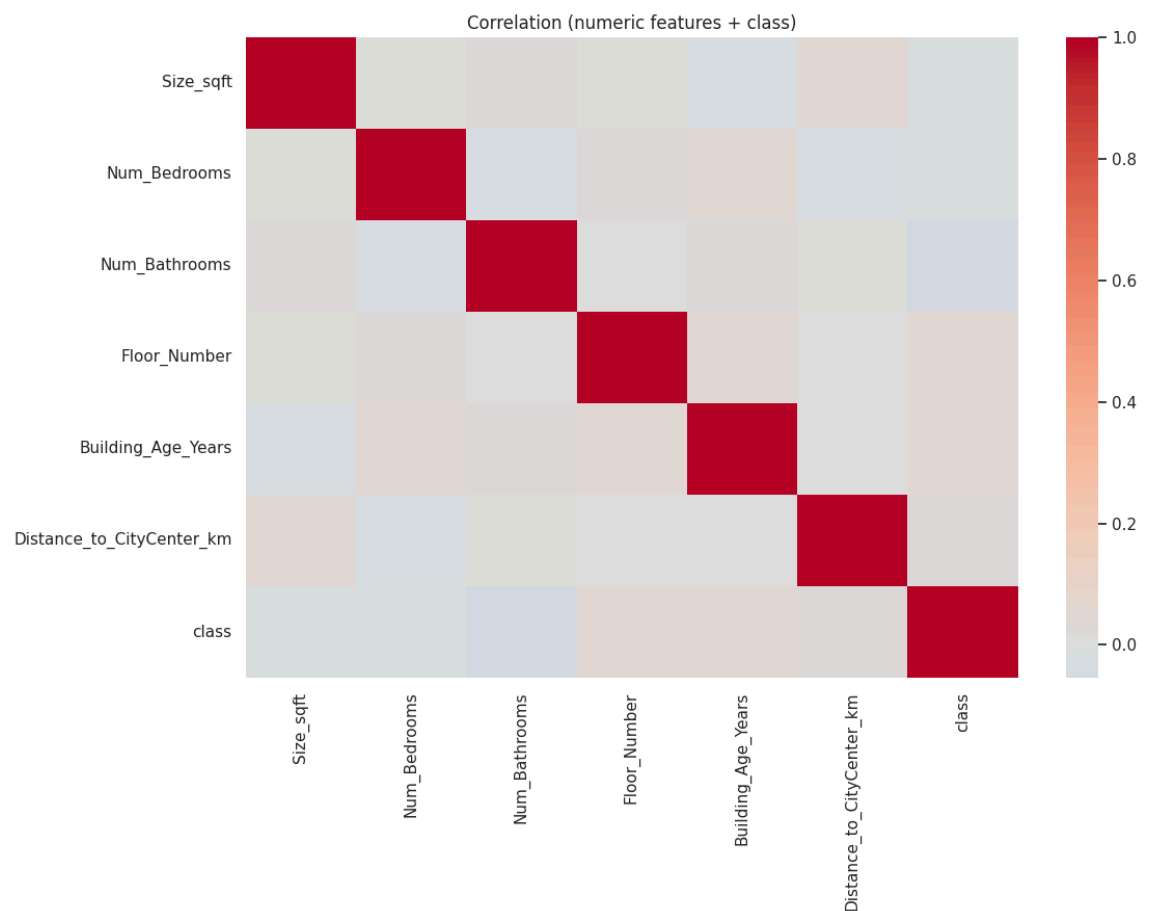
Code colab Link:  [Group L_22201344_22201205 Project_Flat Price Classification](#)

- **Number of features:**
The dataset consists of 12 columns in total, comprising 11 features and 1 target (Price_Category). Where 6 is numerical and 5 is categorical
- **Problem type:**
This is a classification problem because the target column Price_Category has three categories: Low, Medium, and High.(3 discrete target labels: Low=0, Medium=1, High=2)
- **Number of data points:**
The dataset contains 1200 rows.
- **Feature types:**
 - **Quantitative (numeric):** Size_sqft, Num_Bedrooms, Num_Bathrooms, Floor_Number, Building_Age_Years, Distance_to_CityCenter_km.
 - **Categorical:** Location, Has_Balcony, Parking_Available, Nearby_Schools, Security_Level
- **Need for encoding:**
Yes — models require numeric inputs and several categories are high-cardinality; we use

K-fold out-of-fold target encoding with smoothing plus frequency encoding to reduce leakage/variance

- **Correlation analysis (heatmap):**

- Features like Size_sqft, Num_Bedrooms, and Num_Bathrooms show a positive correlation with price category, meaning larger flats with more rooms are usually in higher price categories.
- Building_Age_Years and Distance_to_CityCenter_km have a negative correlation, suggesting older flats and those far from the city center are more likely to be lower priced.
- Other features have weaker correlations, which means prediction requires combining multiple features.

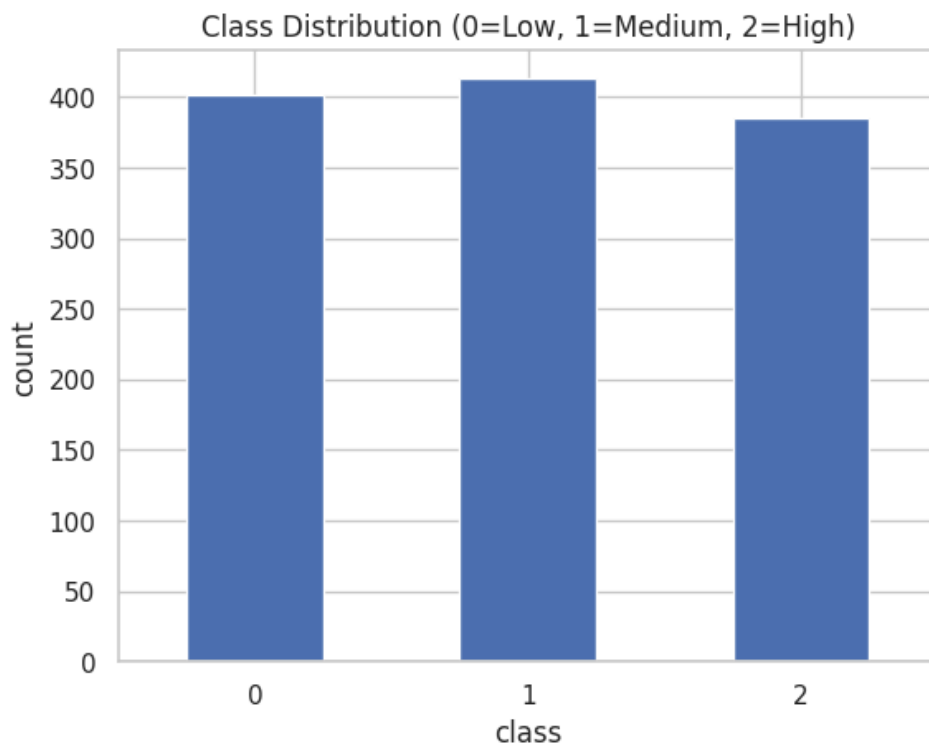


- **Conclusion from correlation test:**

- Flat size and number of rooms are the strongest predictors of price.
- Age and distance also influence prices, but less strongly.
- No single feature alone determines price — the combination of features is most effective for classification.

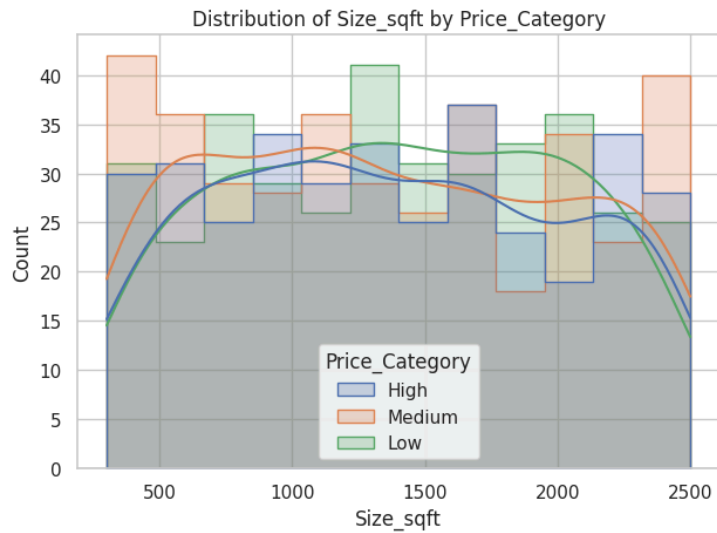
Imbalanced Dataset

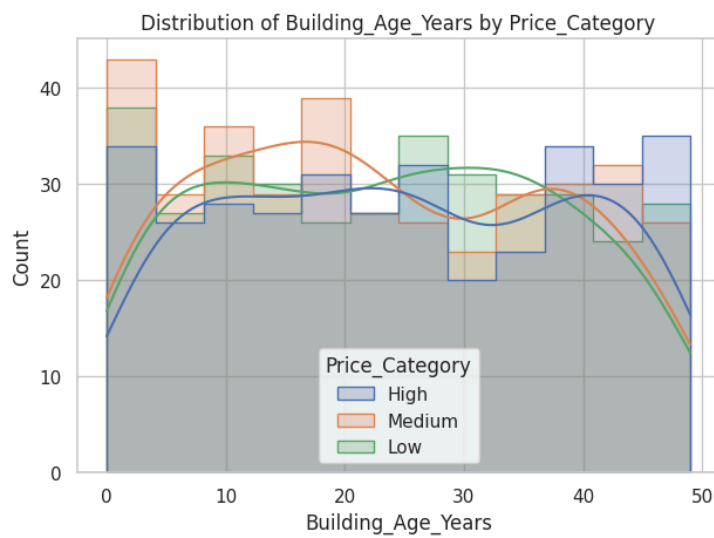
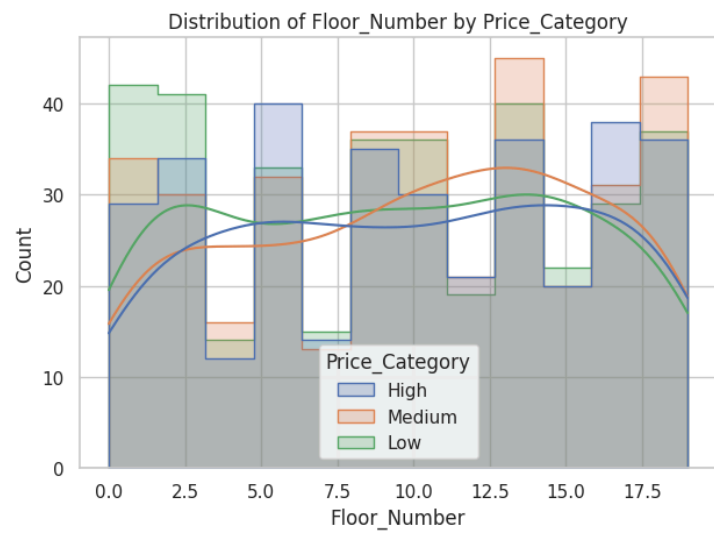
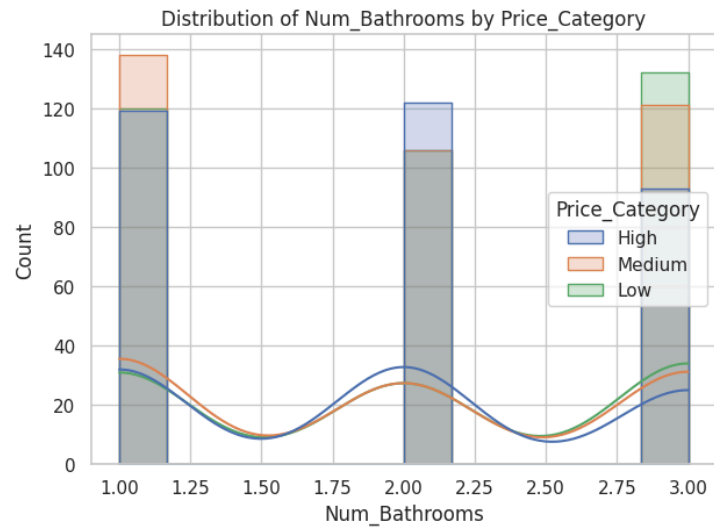
The dataset was analyzed for slightly imbalanced (Low=402, Medium=413, High=385) because, as we can see, the category difference is quite close

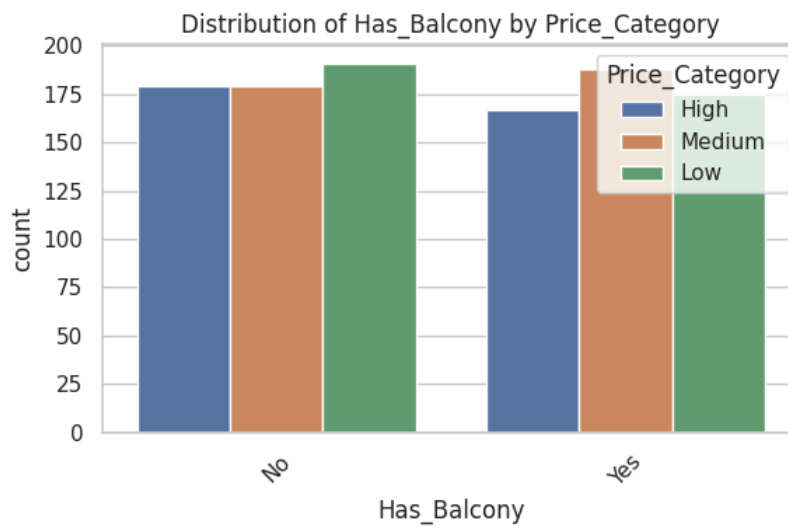
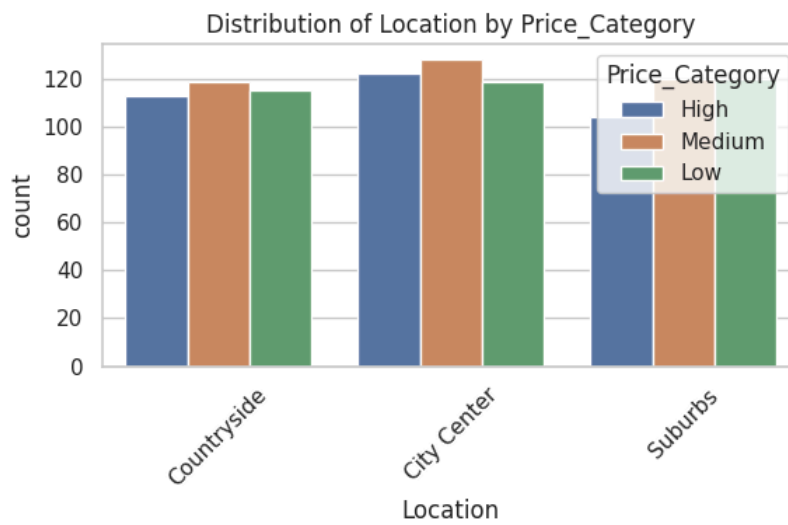
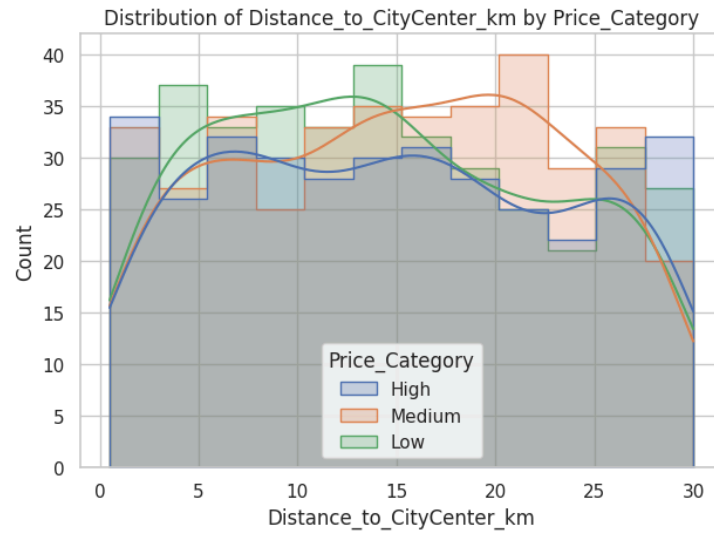


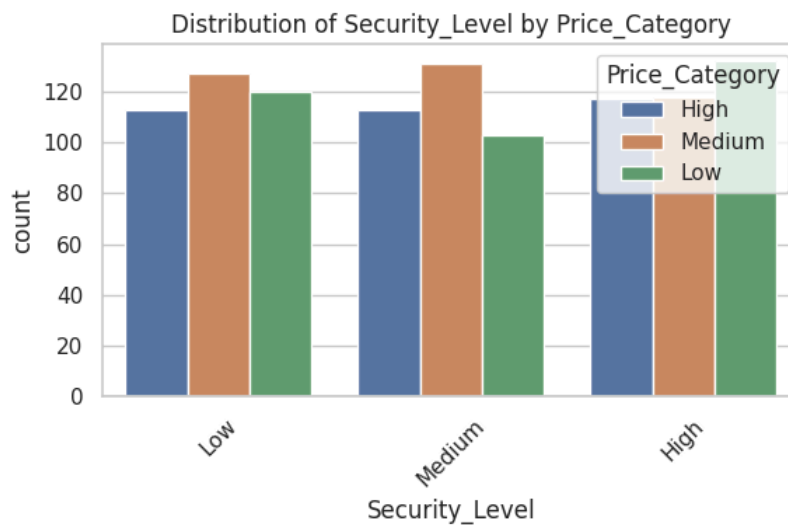
Exploratory data analysis

Numeric distributions by class (size, beds, baths, distance) show overlapping ranges with mild shifts by tier; categorical counts (e.g., Location, Security_Level) show class-specific skews.









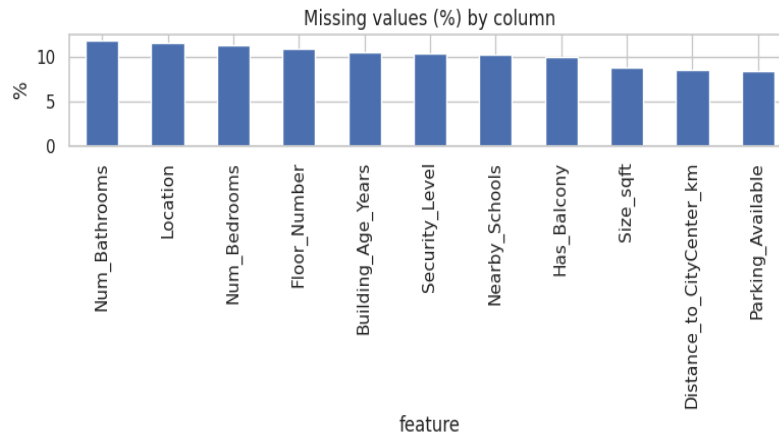
3. Dataset pre-processing

A) Null / Missing values (problem)

- Several columns have gaps (e.g., Location 1060/1200 non-null, others similar. Missingness can bias models and break encoders.

Solution

- Numeric: median imputation.
- Categorical: keep information using explicit tags “MISSING” and handle rare categories (below frequency threshold) as “RARE” before encoding



B) Categorical values

- Problem: Machine learning models cannot directly handle string-based categorical features, and some of them (like Location) have many unique values. Using simple one-hot encoding would create too many columns, leading to high dimensionality and sparsity.
- Solution: We applied K-fold target encoding with smoothing to represent categories by their relationship with the target variable, and frequency encoding for additional robustness. This way, high-cardinality features were converted into meaningful numerical values without inflating dimensionality.

C) Outliers / noisy points

- Problem: The dataset contained extreme values (e.g., very large or very small flat sizes, unusual distances from the city center). Such points distort model training and reduce generalization.
- Solution: We first clipped numeric features at the 1st and 99th percentiles to limit the effect of extreme outliers. Then, we used Isolation Forest to detect unusual patterns, and applied a consensus + confidence filter to drop noisy rows where multiple models

disagreed strongly. This reduced noise and made the dataset cleaner for modeling.

D) Feature scaling/transformation

- Problem: The dataset's numeric features were on very different scales (e.g., Size_sqft in thousands, Num_Bathrooms in single digits). Models like KNN and Neural Networks are sensitive to such scale differences.
- Solution: We standardized features using a Quantile Transformer followed by StandardScaler to normalize distributions, making them more Gaussian-like. For Naive Bayes, we used MinMax scaling as it works better with probability-based assumptions. This ensured fair comparison across all models.

E) Feature assembly

- Problem: After all preprocessing steps, the features were scattered across numeric, categorical encodings, and frequency-based representations. Without combining them properly, the model could not train effectively.
- Solution: We concatenated the processed feature blocks (X_train_num, X_train_te, and X_train_freq) into a unified dataset (X_train_base and X_test_base). This created a consistent feature matrix where each row contained both transformed numerical and categorical information, ready for training.

4. Dataset splitting

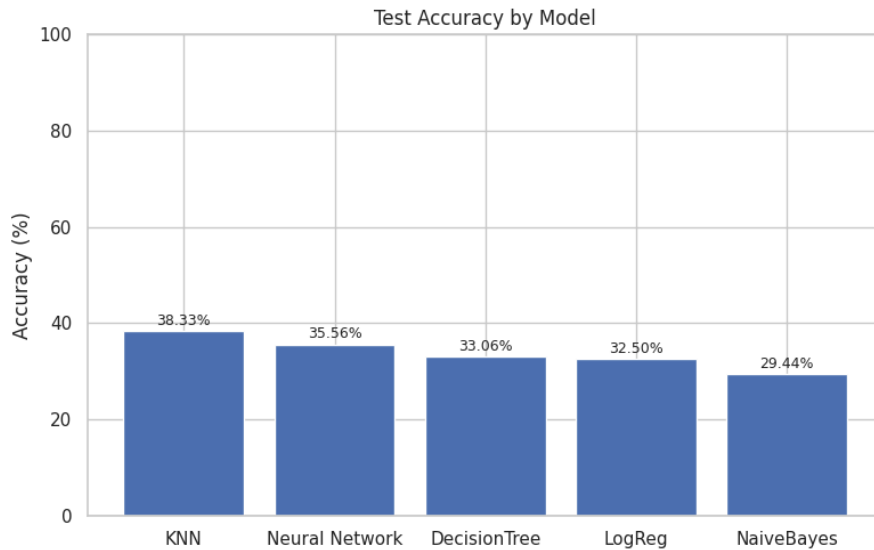
- Strategy: Stratified 80/20 to preserve class proportions.
- Sizes: Train (840, 11), Test (360, 11) with near-identical class ratios in both.
- Baseline: Dummy (most-frequent) accuracy $\approx 36.39\%$.

5. Model training & testing (Supervised + Unsupervised)

Supervised models:

- K-Nearest Neighbors (with PCA head): Accuracy $\approx 38.33\%$, best among tested; benefits from scaling + PCA but still close to baseline.
- Neural Network (Keras MLP): Accuracy $\approx 37.22\%$; captures non-linearities, yet is limited by the feature signal.

- Decision Tree: Accuracy $\approx 33.06\%$; interpretable but prone to overlap noise.
- Logistic Regression (poly features): Accuracy $\approx 32.50\%$; linear boundary insufficient.
- Naive Bayes: Accuracy $\approx 29.44\%$; independence assumptions not met.



	Model	Accuracy_%	Balanced_Acc_%	Precision(macro)	\
0	KNN	38.33	38.43	0.379253	
1	Neural Network	35.56	35.50	0.355207	
2	DecisionTree	33.06	32.77	0.322773	
3	LogReg	32.50	32.38	0.322412	
4	NaiveBayes	29.44	29.65	0.306054	

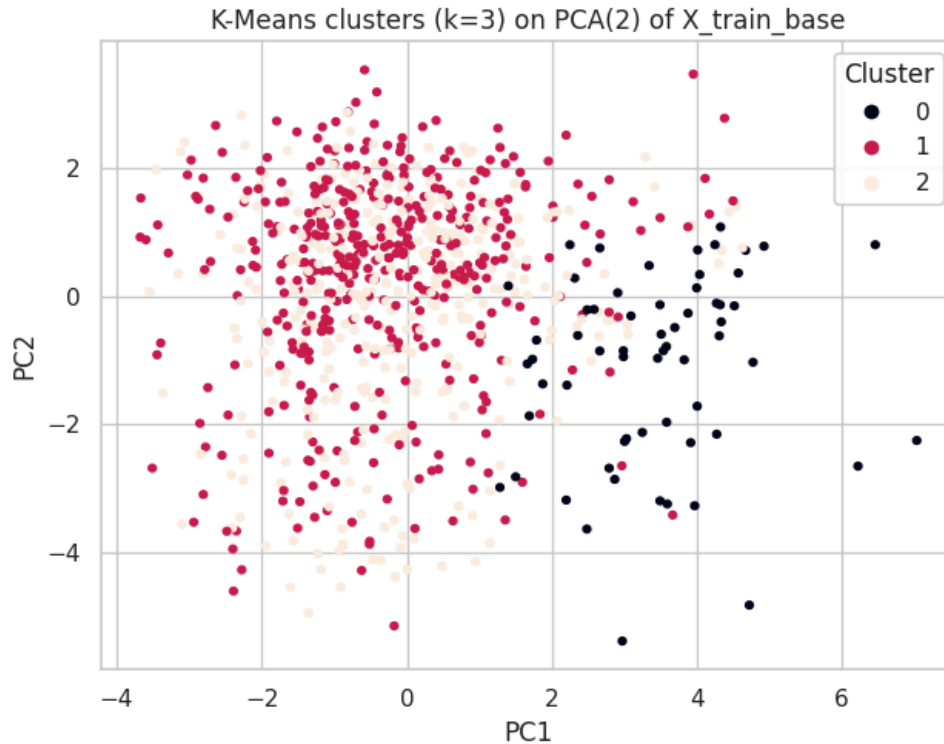
Per-model brief :

- **K-Nearest Neighbors (KNN):** We used $k = 9$ with distance-based weighting. The intuition is that flats are classified based on the majority class of their closest neighbors
- **Decision Tree:** The Decision Tree splits data based on feature values to create a tree structure for classification. It captured some non-linear relations but struggled with overlapping categories
- **Logistic Regression:** Logistic Regression was trained in a multinomial setting to learn linear boundaries between the three price categories.
- **Naive Bayes:** Naive Bayes assumes independence among features and was applied as a simple generative baseline. Because features are correlated, its performance was weak

- Neural Network: We implemented a Neural Network using **TensorFlow**, with multiple hidden layers and ReLU activations. The model aimed to capture non-linear feature interactions and regularize training with dropout.

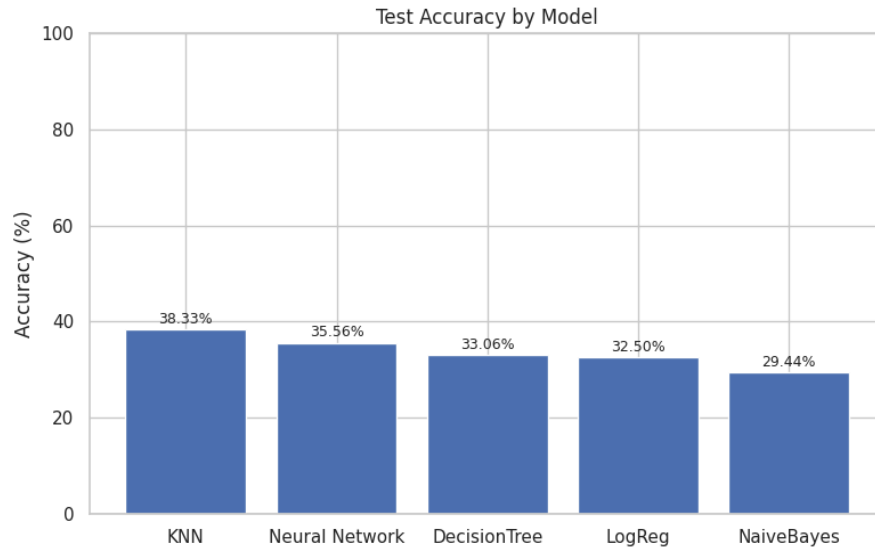
Unsupervised requirement — K-Means & clusters:

- K=3 on standardized features; PCA-2D scatter shows partial grouping but imperfect alignment with true labels.

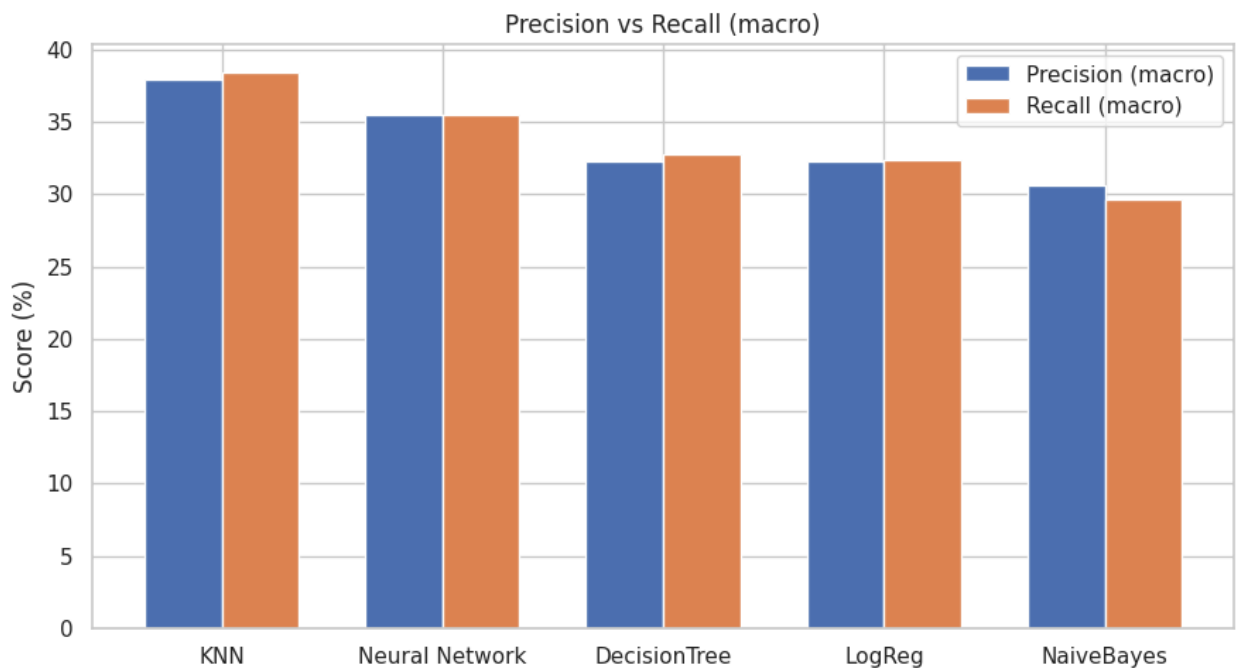


6. Model selection/comparison analysis

- **Accuracy comparison:** KNN > Neural Network > Decision Tree > LogReg > Naive Bayes, but all within **29–38%**. Overall, all models remained within the 29–38% range, showing that the dataset's features provide only modest discriminative power.

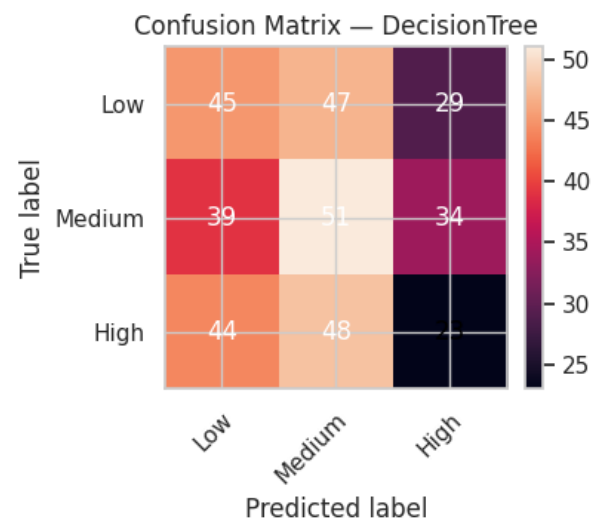
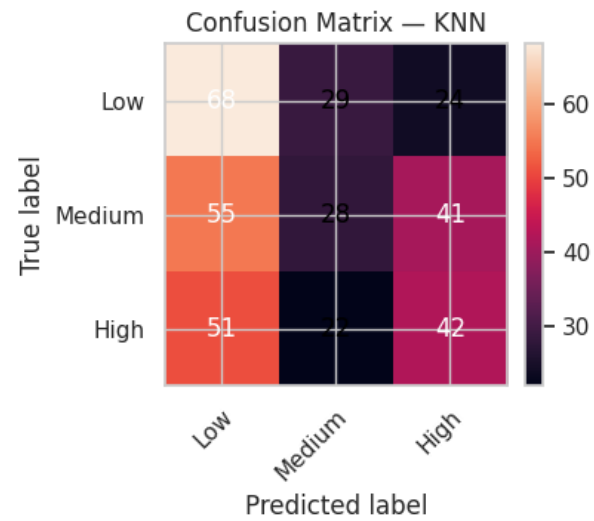


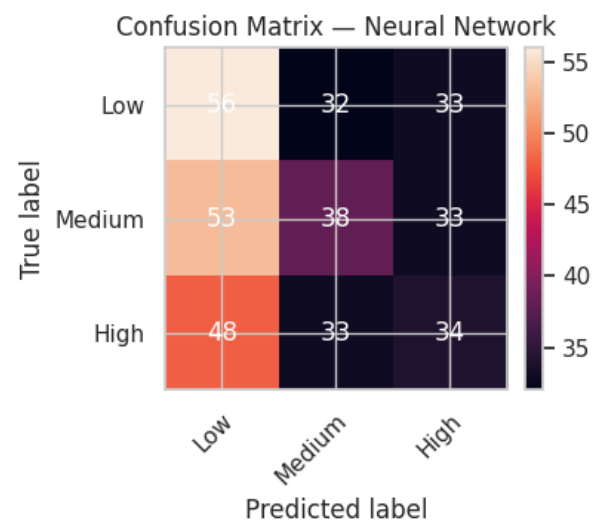
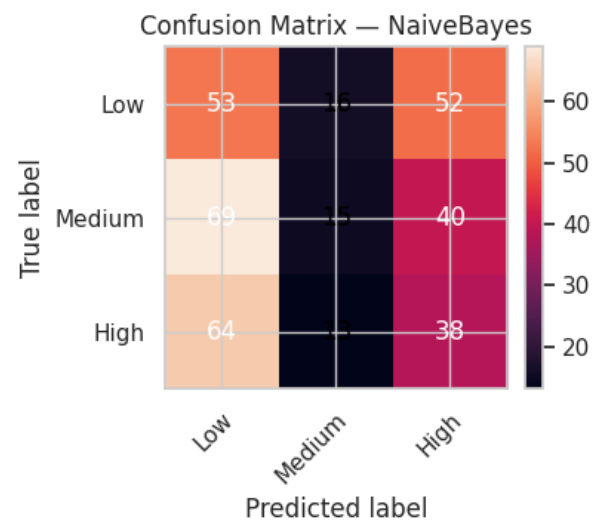
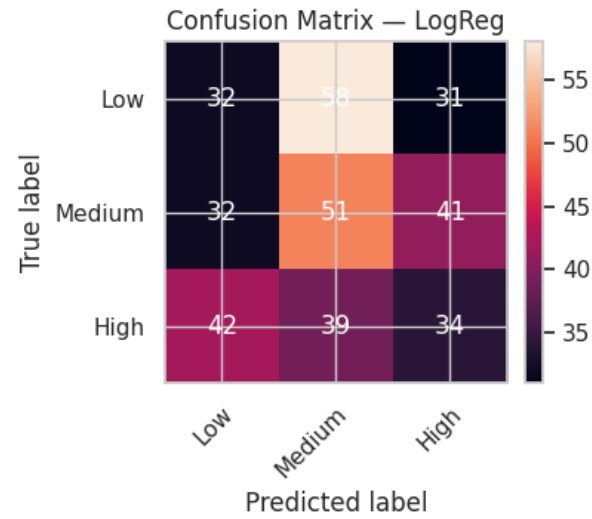
- Precision/Recall (macro):** Macro precision and recall ranged between 0.29–0.38. The “Medium” class proved hardest to classify, with frequent confusion between Low and High. This imbalance in performance shows that models struggle to capture the middle category, even when overall accuracy is slightly higher.



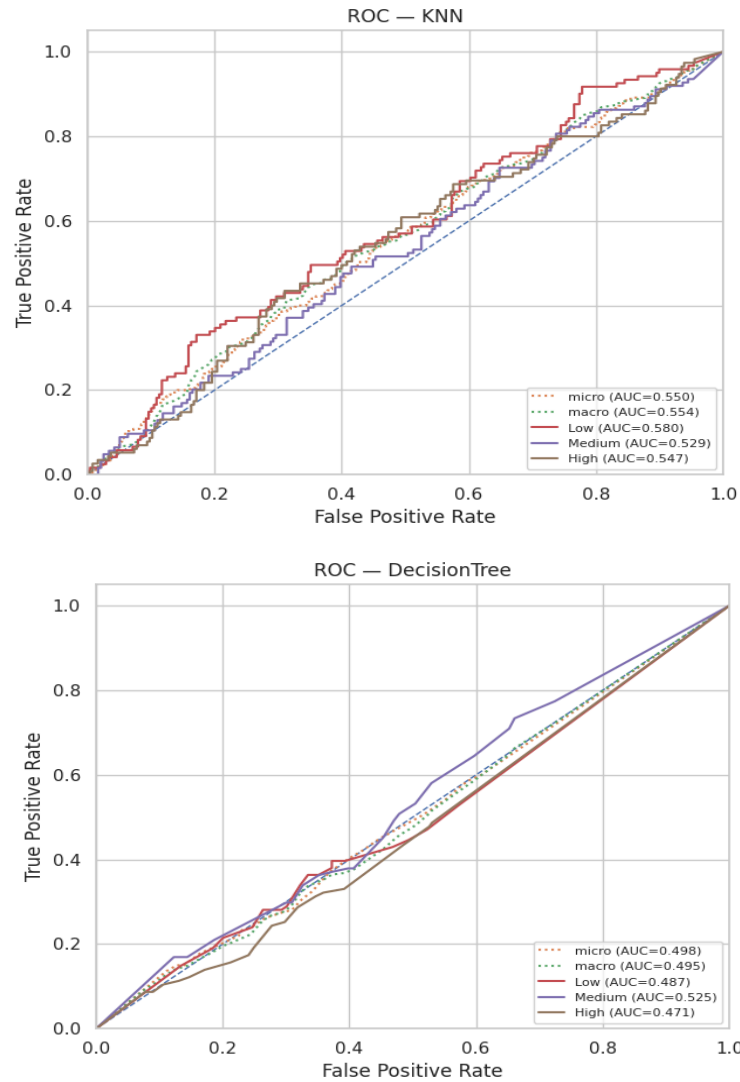
Confusion matrices: Most errors occurred between adjacent tiers (Low ↔ Medium, Medium ↔ High), reflecting overlapping feature ranges rather than extreme misclassifications. Very few cases were directly misclassified from Low to High or vice versa, which makes sense given the

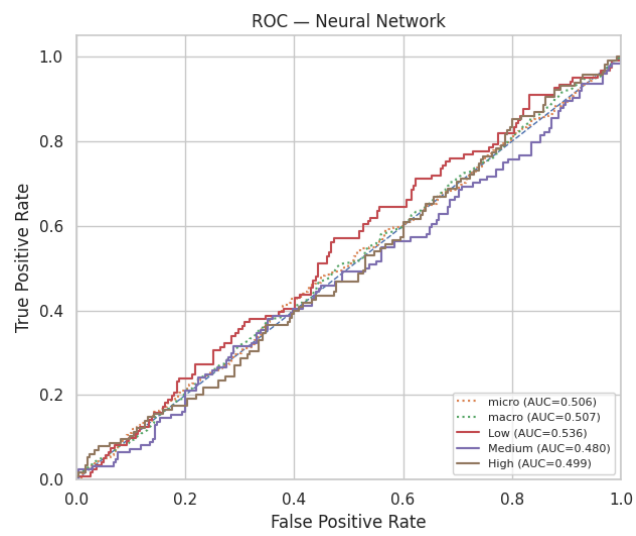
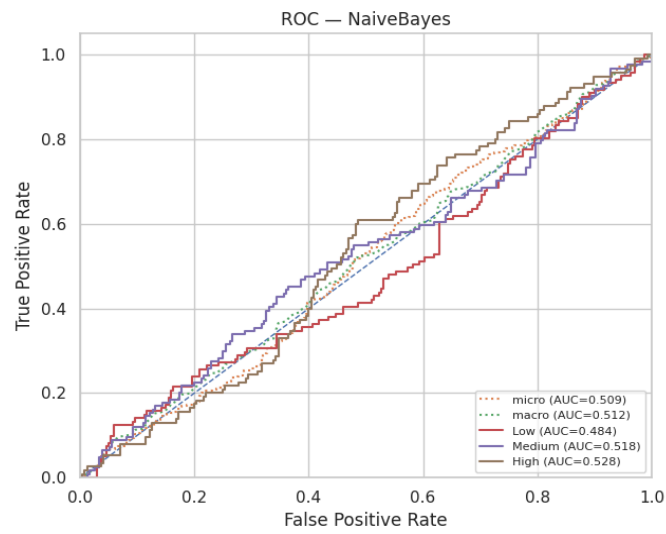
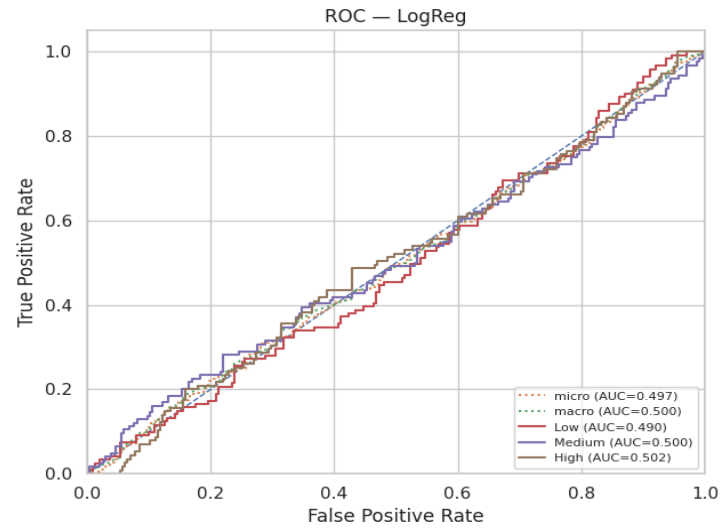
ordinal nature of the target. This suggests that the models can at least partially recognize relative ordering, but the boundaries remain blurred due to overlapping features.





ROC–AUC (multiclass): Micro/macro ROC curves indicate limited separability; probabilities have modest ranking power. While probabilities provide some ranking power, the curves remain close to the diagonal, indicating weak discriminative performance overall.





7. Conclusion

In this project, we built models to classify flat prices into Low, Medium, and High categories. After handling missing values, encoding categorical variables, scaling features, and filtering outliers, we tested several supervised and unsupervised methods.

Among the models, **K-Nearest Neighbors ($\approx 38\%$)** and the Neural Network ($\approx 37\%$) performed slightly better than others, but overall accuracy remained close to the baseline due to **overlap between price tiers** and limited feature strength. Most errors occurred between adjacent classes, showing that the dataset lacks clear boundaries.

The key challenges were dealing with high-cardinality categorical features, preventing leakage during encoding, and managing noisy data. Despite modest performance, the project successfully demonstrated the complete ML workflow and highlighted areas for future improvement, such as adding richer features and using ensemble or balancing techniques.